Edge Sensitivity at Package Level: Report

CMPE 275, Mini 1.

Vandan Shah (018521672)

Neeraja Buch (018178238)

Dhruv Jain (019150859)

1. Summary

We built a C++ library that loads the EPA Air Quality System hourly ozone and nitrogen dioxide files (12 CSV files, 65,742,181 readings, 15.5 GB) and answers four kinds of searches over them, and we used it to study how three design choices change speed and memory: the layout of the data in memory, the boundary between a library and the application that calls it, and threads. The work had two phases. In Phase 1 we wrote a serial, object-oriented library and measured it on the full dataset. In Phase 2 we first applied OpenMP to that design and then rewrote the data classes as an optimized library linked to a separate application, measuring each change on its own. Our main findings are these. Parsing, not input or storage, dominates the load: splitting lines into fields took about 78% of the load time (79.7% and 77.4% in two profiles), and a single-pass tokenizer cut the serial full-dataset load from 23.6 s to 10.3 s (2.3 times faster) with identical results and memory. A column layout makes searches that read only one field up to 4 times faster than the Phase 1 baseline but makes searches that must return whole rows no faster and often slower (copying one monitor's year of readings was about ten times slower), so the best layout depends on what the query returns. The library boundary and static versus shared linking cost nothing we could measure. Threads helped, but at a cost: eight threads made the load 3.5 times faster than the same code on one thread, yet only 1.5 times faster than the serial load with the new tokenizer, and they needed 2.6 times the memory; for searches the serial column count (2.7 ms) beat the OpenMP count on the row layout at every thread count (5.7 ms at best), although part of that gap is the loop, since the OpenMP code still uses the original comparison loop. Finally, the resident-memory figure that standard tools report on macOS was badly wrong for this workload, so we measured the physical memory footprint instead (Section 4.2).

All timings in Sections 4 to 8 come from one machine, a MacBook Air with an Apple-silicon processor (4 performance and 6 efficiency cores, 16 GB, plugged in), built with Homebrew Clang 23.1.3 and CMake in Release mode. We call it M-A. Dhruv Jain's measurements come from a different machine (Intel Core i9-13900HX, 32 logical processors, 15 GiB visible to WSL2 under Windows 11, GCC 13.4.0), which we call W-1. We never put M-A and W-1 numbers in one table or graph, because the two machines differ in many ways; where both measured the same change we compare the ratios and say so.

2. Problem, data and design

The data are the EPA AQS pre-generated hourly files for ozone (parameter 44201) and NO2 (42602), one file per pollutant and year [1][2]. A monitor is one instrument, identified by state, county, site number, parameter code and instrument number; the dataset holds 1,941 of them, of which 1,283 ozone and 473 NO2 monitors reported in 2024. Four searches are supported. Q1 returns one monitor's readings in a time range. Q2 returns all readings of a pollutant whose value lies in a range, optionally within a time range. Q3 returns the count, minimum, maximum and mean of a pollutant over a time range, grouped by

nothing, by monitor, or by hour of day. Q4 lists monitors in a state or in a latitude and longitude box. The assignment also asks how results should leave the library, so the searches have several result modes: Q1 can return a copy of the rows, a zero-copy view, or (for comparison) be answered by a scan without the per-monitor index, and Q2 can return a count, a copy of the matching rows, a batched callback, or one virtual call per row.

The Phase 1 design is deliberately simple. One reading is a 12-byte record: a 16-bit monitor identifier, a 32-bit hour counted from 2021-01-01, a 16-bit value, and two 8-bit codes (80 bits of fields, padded to 12 bytes by alignment). Values are stored as exact integers (ozone in parts per million times 1,000, NO2 in parts per billion times 10), which matters for correctness, as shown in Section 4. Everything that is the same for all readings of a monitor (codes, location, names) is stored once in a monitor registry, and the qualifier and method codes are stored once in dictionaries. All rows live in one array with two indexes: per-file segments, which let a search skip files whose time range does not overlap, and per-monitor blocks, which allow a binary search on time within a monitor. A facade class is the only thing the driver program uses. We kept three file readers (a naive stream reader, a getline reader and a buffered block reader) so that they could be compared. The whole dataset fits in memory as 810 MB, about 12.3 bytes per reading, against 15.5 GB of CSV.

Correctness was checked independently of the C++ code: a Python script over the raw CSV files gave the same totals as our full-dataset test (65,742,181 rows, 0 bad rows; 1,283 ozone and 473 NO2 monitors in 2024; 3 NO2 readings above 100 ppb in 2024; 62,074 ozone readings above 0.070 ppm in 2024). The test suites pass in Release and under AddressSanitizer and UndefinedBehaviorSanitizer, and the full-dataset test passes on both machines.

3. Method

Loads were timed as whole processes with one untimed warm-up and then ten timed runs, run one after another with the machine kept awake. The final Phase 2 benchmark run in Sections 5 to 7 was taken on mains power, with two exceptions that we label where they appear: the OpenMP load scaling in Table 3 and the checks in Section 9 were taken on battery power (charge between 79% and 96%), with the builds interleaved so that the comparisons within each are valid. The earlier Phase 1 baselines in Section 4 were taken with the machine awake, but we did not record whether it was plugged in. Searches were timed inside one process: the data were loaded once and each query was repeated ten times (twenty in Dhruv Jain's W-1 comparisons of narrow time ranges), and we report the minimum, because the minimum is the least affected by other activity. Because a search process must load the dataset first, whole-process timing would measure the load, not the search. When two builds had to be compared on load time we alternated them round by round (interleaved runs) so that slow drift in the machine affected both equally, and we report medians. We benchmarked Release builds only. Memory is reported as the peak physical memory footprint described in Section 4. Every number below comes from our own runs; we do not use published figures for our results.

Early in Phase 2 we compared two builds that differed by about 4% in load time and could not decide whether the difference was real. Running the builds interleaved on mains power gave medians of 23.59 s, 23.57 s and 23.66 s for the three variants, so the 4% was drift, and a sequential comparison of this kind should not be trusted on this machine. A second lesson is that the first benchmark run of the Phase 2 searches began with the laptop on battery power and was plugged in part-way; it produced at least one outlier (a 126 ms callback time that later measured about 90 ms), so we repeated the full set of searches on mains power and use that run in Sections 6 to 8, except where we say otherwise.

4. Phase 1 results

4.1 Loading

Table 1 and Figure 1 show the full-dataset load on M-A for the three readers, with and without pre-sizing the row array from the file sizes. The buffered reader took 22.95 s, the getline reader 23.90 s (4% slower) and the naive reader about 43.5 s (1.9 times slower than buffered). A first measurement on the 2024 files alone, made on a different machine (16.60 s against 8.56 s), gave a ratio of 1.94, so the ratio is stable across data sizes and machines even though the absolute times are not comparable. The naive row in Table 1 has a mean inflated by two slow runs (52.4 s and 54.3 s; the other eight runs were 43.4 to 43.8 s). The machine is a fanless laptop, so thermal throttling or background activity is plausible, but we did not prove it, and we kept both runs in the statistics. Pre-sizing costs nothing in time and halves the peak memory (801 MB against 1,606 MB), because without it the row vector grows by doubling and the old and new buffers exist at the same time.

Reader	pre-sized	mean (s)	sd	min	max	peak footprint
buffered	yes	22.95	0.79	22.58	25.16	801 MB
buffered	no	22.91	0.14	22.69	23.12	1,606 MB
getline	yes	23.90	0.15	23.73	24.15	792 MB
getline	no	24.12	0.11	24.00	24.35	1,598 MB
naive	yes	45.49	4.17	43.36	54.26	793 MB
naive	no	43.48	0.22	43.27	44.02	1,598 MB

Table 1. Phase 1 full-dataset load, M-A, ten timed runs each after one warm-up. Footprints are in MB (10^6 bytes); Figure 1 and the later tables use MiB (2^20 bytes), so 801 MB is 764 MiB.

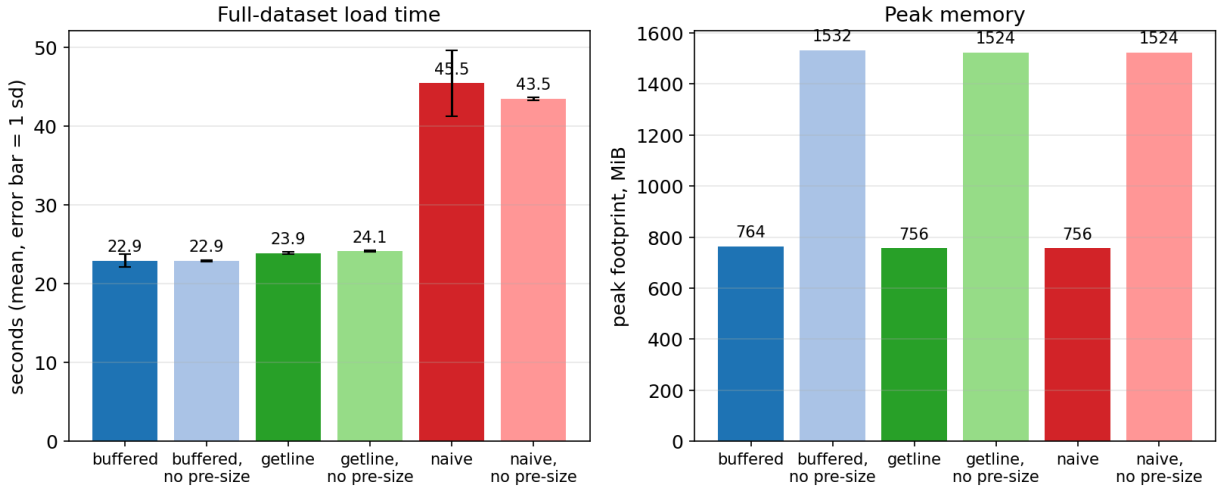


Figure 1. Phase 1 full-dataset load on M-A: mean time with one standard deviation (left) and peak physical footprint (right) for each reader, with and without pre-sizing the row array.

To find where load time goes we sampled a 2024 load with the macOS `sample` profiler (2,553 samples). Splitting lines into fields (calls to `memchr` and the tokenizer) accounted for 79.7% of the samples, number and date parsing for 9.1%, reading the file for 6.0%, and storing rows for 0.9%. We repeated the profile at the end of the project and got 77.4%, 11.9%, 7.8% and 1.3% (2,501 samples), so the splitting share is

about 78% and the other shares move by a few points between profiles. The splitter called find twice per field, about 600 million short searches for 2024 alone. This was the evidence for the single-pass tokenizer in Section 7. We measured Phase 1 twice: Table 1 is the original baseline (buffered reader, mean 22.95 s), and the interleaved comparison used in Sections 6 to 8 gave a mean of 24.15 s and a median of 23.61 s on a later day. The difference is within the variation we saw between sessions, and Sections 6 to 8 use the interleaved figure because it was taken alongside the Phase 2 builds.

4.2 The memory measurement problem

Our first memory numbers were wrong. The usual measure, peak resident set size from getrusage, reported 80 to 149 MB for a structure that is about 810 MB, across seven identical runs (three when we first noticed the problem, 80 to 116 MB, and four when we re-checked at the end, 88 to 149 MB), a spread of 70 MB. The /usr/bin/time figure was also unreliable: it reported 95.5 MB in the first of ten identical runs and about 801 MB in the other nine, and for the run without pre-sizing it reported 1.2 to 1.4 GB against a true 1.6 GB. Resident size on macOS leaves out memory the operating system has compressed, so it depends on how much compression has happened at that moment. We replaced it with the peak physical memory footprint, read with task_info and the TASK_VM_INFO flavor (the ledger_phys_footprint_peak field) [3], the figure shown by Activity Monitor. It agreed to within 1 MB across all of those runs (800.8 to 801.7 MB) and about 1% below the size of the store. The error is also not a constant factor, so it cannot be corrected afterwards: in the std::string experiment of Section 4.4 the in-process resident figure was 4.32 GB, 23% below the 5.62 GB reported by /usr/bin/time for the same process, and 3.07 GB, 45% below, when we repeated it, so it under-reports even at multi-gigabyte scale. Every memory figure in this report is footprint, and we kept the old figure in the program output unchanged for comparison.

4.3 Searches

Table 2 gives the Phase 1 search times. A Q1 lookup of one monitor’s three readings took 83 ns when copied and 41 ns as a view, against 15.2 ms for a scan without the per-monitor index, so the index makes the view lookup about 370,000 times faster. The view stays at 41 to 42 ns whether it returns 3 or 8,676 rows because it returns pointers into storage. For Q2, count, copy and callback cost about the same while few rows match, because all three scan the whole pollutant once, and the count costs 11.3 ms whether 0.6% or 54% of rows match. At 54% selectivity copying 25.3 million rows (303 MB) adds about 46 ms to the scan (57.8 ms against 11.3 ms), so at high selectivity the way results leave the library, not the scan, dominates the cost. A virtual call per row was about 3 to 9 times slower than the templated scans when few rows matched. Q3 grouped by monitor was six times slower than ungrouped (25.1 ms against 4.2 ms), which we attribute to random access into about 1,300 accumulators instead of one; we did not test that explanation separately. The Q2 search for all readings in one hour without naming a monitor (a “time-only” search) took 3.80 ms, because rows are ordered by monitor and then time, so only whole files can be skipped. This is the number the indexed search in Section 7 had to beat.

Q2 search	count	copy	callback	virtual	rows returned
NO2 > 100 ppb	4.49 ms	4.67 ms	4.69 ms	13.07 ms	38
ozone > 0.070 ppm	11.36 ms	13.03 ms	13.09 ms	33.87 ms	281,349 (0.60%)
ozone > 0.030 ppm	11.34 ms	57.83 ms	62.62 ms	62.79 ms	25,272,790 (53.8%)

Q2 search	count	copy	callback	virtual	rows returned
ozone, one hour (time-only)	3.80 ms	3.80 ms	3.80 ms	34.27 ms	1,205

Table 2. Phase 1 Q2 times on M-A, minimum of ten in-process repeats.

4.4 Two experiments on representation

We tested two alternative representations on the 2024 ozone file (8,995,978 rows). Storing every field as a `std::string` used 624 bytes per row, which extrapolates to 41 GB for the whole dataset against 810 MB for the compact record (about 52 times larger), so it cannot be loaded on a 16 GB machine (a repeat of the single-file run reported a peak physical footprint of 10.1 GB, more than the 5.6 GB maximum resident size, so the extrapolation from counted bytes is if anything an underestimate); it took 9.4 to 9.6 s for the single file (two runs). Storing values as `float` is harmless for rounding to the displayed decimals, but the test `value > 0.070` was wrong for every one of the 9,348 readings of exactly 0.070 ppm: all of them counted as above the threshold, while `double` and our scaled integers got none wrong. Our explanation, which we inferred and did not verify in the code, is that the nearest `float` to 0.070 is slightly above the `double` 0.070 it is compared with. This is a correctness argument for exact integer storage: 0.070 ppm is the level of the current U.S. 8-hour ozone standard (our readings are hourly, so this illustrates the risk and is not a compliance calculation), and a count of readings above a round threshold is exactly the kind of question these data are used for.

5. Phase 2, step 1: OpenMP

Dhruv Jain applied OpenMP to a separate copy of the Phase 1 design (`part-b/omp`), so the baseline stayed untouched. The Q2 count is a parallel reduction. Q2 copy writes one result vector per input file and merges them in file order, so that results are identical to the serial order. Q3 gives each thread its own accumulators and merges them serially. Q1, the batched callback and the per-row virtual call stay serial: Q1 takes well under a microsecond, so starting threads would cost more than the query, and the callback interfaces are sequential by definition. Loading parses the files in parallel. Each file has its own private registry, dictionaries and row vector, and the main thread merges them in file order and renumbers the local identifiers, so the stored rows, monitors and dictionaries are identical to a serial load at every thread count. File-level parallelism already scaled well, so we did not split individual files into byte ranges.

An early attempt failed and taught us something. With the default contiguous schedule, the six ozone files all went to one thread and the six NO2 files (which a search skips immediately) went to the other, so a two-thread search showed no speedup at all. Scheduling with `schedule(static, 1)`, which deals files out round-robin, fixed it [4]. Because only six ozone files exist, a Q2 or Q3 search has at most six units of work, which explains why speedup stops near six threads.

Table 3 and Figure 2 give the scaling on M-A. The OpenMP load takes 23.9 s on one thread, equal to the serial Phase 1 load because the parsing code is the same, 6.6 s on eight threads (3.6 times) and 6.1 s on ten (3.9 times); these are medians of ten interleaved rounds, taken on battery power at 86% charge. An earlier interleaved session on mains power, which also included Phase 1, gave 24.06, 8.70 and 6.94 s at 1, 4 and 8 threads (3.5 times at eight), within 6% of this one, and Sections 1 and 8 use those figures because Phase 1 was measured in the same session. Searches reach 2.8 times at six threads for the count and 2.7 times for Q3. This is below the 3.7 to 3.8 times Dhruv measured on W-1 (Table 4). Parallel efficiency on M-A falls from 73% at four threads to 51% at six, which is consistent with the machine having four performance cores and six slower efficiency cores, so threads beyond four do not add equal capacity;

unlike W-1, the load still gains a little from eight to ten threads. Memory is the cost: the OpenMP load peaks at 1,643 MiB on one thread and 2,113 MiB on eight, against 764 MiB for the serial loaders, because each file's rows exist twice, once in its private vector and once in the merged store, and more buffers are live at once with more threads.

Threads	load, median (s)	speedup	efficiency	peak footprint	Q2 count (ms)	Q2 copy (ms)	Q3 (ms)
1	23.88	1.00	100%	1,643 MiB	16.3	15.7	24.6
2	13.70	1.74	87%	1,753 MiB	8.2	9.9	15.6
4	8.18	2.92	73%	1,874 MiB	7.5	8.5	11.1
6	7.78	3.07	51%	2,095 MiB	5.8	6.8	9.0
8	6.61	3.61	45%	2,113 MiB	6.9	6.3	10.1
10	6.11	3.91	39%	2,130 MiB	5.7	6.8	10.7

Table 3. OpenMP scaling on M-A. Loads: ten interleaved rounds in one session on battery power (86%). Searches: ozone data, minimum of ten repeats, measured in the final mains-power run. An earlier mains-power session gave load medians of 24.06, 8.70 and 6.94 s at 1, 4 and 8 threads and footprints of 1,540, 1,874 and 1,966 MiB.

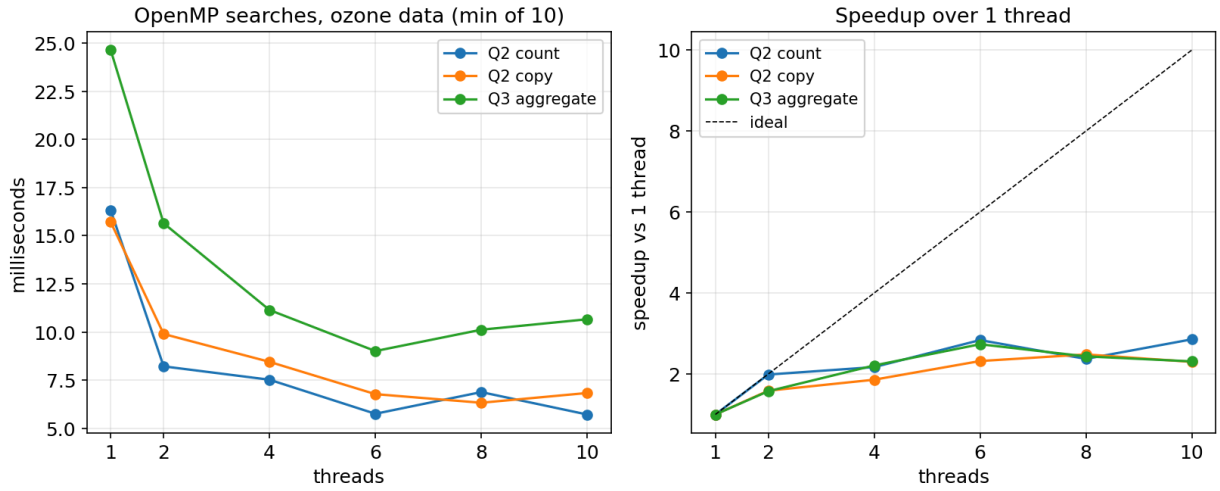


Figure 2. OpenMP searches on M-A over the ozone data: time (left) and speedup over one thread (right), minimum of ten repeats. The dashed line is ideal speedup.

On W-1, loads at 1, 2, 4, 6, 8 and 10 threads took 43.4, 23.5, 12.7, 9.2, 7.3 and 7.4 s, a speedup of 5.95 times at eight threads (efficiency 74%); it stops improving after eight threads while memory rises. The W-1 searches on ozone data, Q2 count 49.3 ms at one thread against 13.4 ms at six (3.7 times) and Q3 52.0 ms against 13.7 ms at ten threads (3.8 times), show the same plateau at about six threads. Table 4 collects these.

Threads (W-1)	load (s)	speedup	Q2 count (ms)	Q2 copy (ms)	Q3 (ms)
1	43.42	1.00	49.3	58.3	52.0
2	23.53	1.84	29.5	35.4	31.1
4	12.65	3.43	20.5	25.8	21.6
6	9.20	4.72	13.4	15.3	14.3
8	7.30	5.95	13.5	15.4	14.6

Threads (W-1)	load (s)	speedup	Q2 count (ms)	Q2 copy (ms)	Q3 (ms)
10	7.45	5.83	14.3	17.2	13.7

Table 4. OpenMP scaling on W-1 (a different machine; compare shapes, not absolute times with Table 3).

Counts and copy checksums matched the serial results at every thread count measured. In addition, the five Phase 1 tests pass at one and four threads, the full-dataset test passes at one and eight threads, and AddressSanitizer and UBSan pass. ThreadSanitizer did not give a clean result, which we discuss in Section 9.

6. Phase 2, step 2: library, layouts and linking

Neeraja Buch rewrote the data classes as a library, aqdata, and a separate application that links to it. The public headers expose only value types, load options, a read-only column view and one Database class. The class uses the pimpl idiom [5]: its data members are a single pointer to an implementation defined in a private source file, so the application cannot include or depend on the store, loader, tokenizer or registry. The library is compiled with hidden symbol visibility, and only the public functions are marked for export. Checking the shared library with nm shows exactly 46 exported functions and none of the internal classes. It can be built statically or as a shared library from the same sources.

We then added a second storage layout behind the same internal interface. The row layout is the Phase 1 array of 12-byte records. The column layout keeps one array per field (hour 4 bytes, monitor 2, value 2, qualifier 1, method 1, 10 bytes per row), with the same per-file and per-monitor indexes, so the only difference between the two is the layout [6][7]. The layout is chosen at run time, which let us benchmark both with one binary. A Q1 view returns strided pointers into the column arrays, so it is zero-copy for both layouts. Correctness checks include a test that loads the 2024 data into both layouts and compares every Q1, Q2 and Q3 result mode row for row (1,147 checks), and a test run that reproduces the full-dataset totals in both layouts.

6.1 The library boundary and linking cost nothing measurable

In the first Step 2 benchmark run the row layout inside the library matched the Phase 1 code to within 2.2% on every search that takes at least 1 ms, except the ungrouped aggregate, which was 9% faster (3.80 ms against 4.18 ms). We repeated the check later with the builds interleaved over four rounds on the same ozone data (Section 9): the library matched Phase 1 to within 2% for the 25-million-row copy (1.02), the per-row virtual call (1.00) and the ungrouped aggregate (0.98), and was 5% slower for the batched callback. The extra layer (a pointer indirection and a function call per query) is therefore not visible in speed, which is expected because the library is entered once per query and the cost is in the scan. The shared library against the static one gave a median ratio of 1.001 for the row layout and 0.993 for the column layout across the 21 searches that take at least 1 ms in the first Step 2 run, with individual ratios between 0.98 and 1.03 for rows and between 0.88 and 1.06 for columns that we attribute to noise (Figure 3). The four nanosecond- and microsecond-scale Q1 cases are left out because the timer resolution of about 40 ns makes their ratios jump between 1 and 2. That run began on battery power and was not repeated, so we treat the ratios as consistent with no difference rather than as a precise measurement. Symbol resolution is not on any hot path, so we found no reason to prefer one linkage for speed; the choice is about deployment.

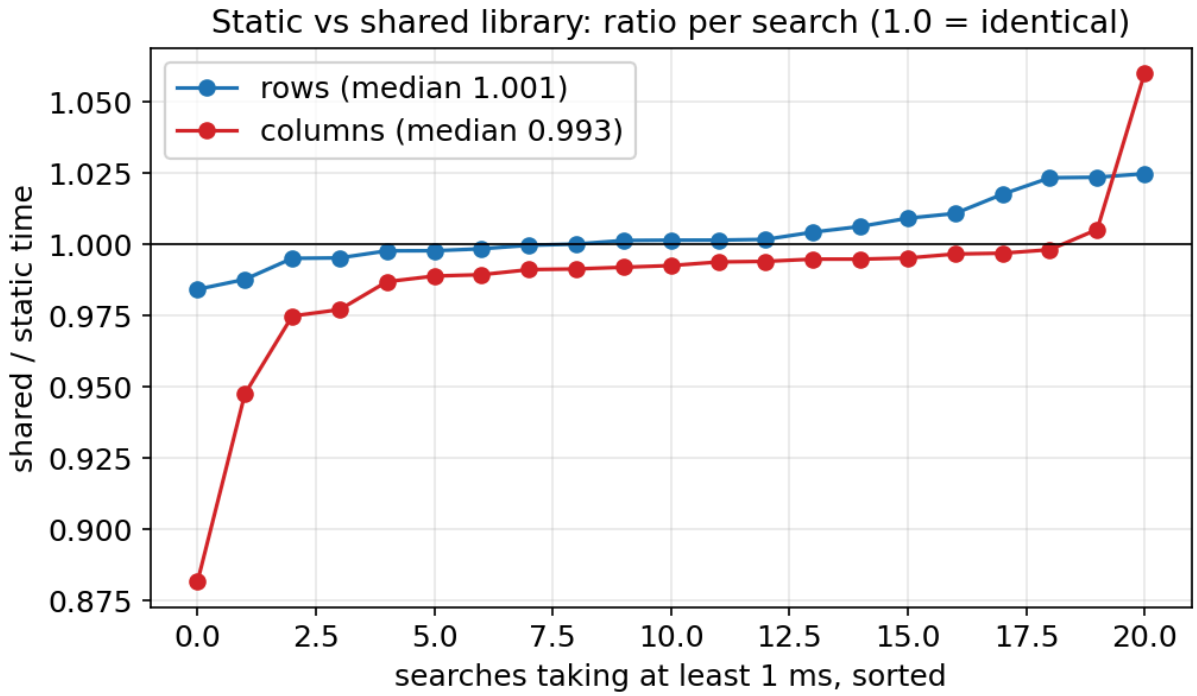


Figure 3. Ratio of shared-library to static-library time for each search that takes at least 1 ms, sorted (1.0 means identical), from the first Step 2 run.

6.2 Column layout: faster for narrow queries, slower for wide ones

The column layout uses 16% less memory (669 MB against 801 MB at the time of that measurement, 638 against 764 MiB in the final one) and loads as fast as the row layout (interleaved medians 10.28 s and 10.32 s, Figure 5). Table 5 and Figure 4 show the searches. A count over values reads only the 2-byte value column, 6 times fewer bytes than the 12-byte rows, and the layout made it 2.7 times faster than the same loop on rows (2.7 ms against 7.4 ms) and about 4 times faster than the Phase 1 count (11.4 ms). A scan for a monitor without an index that only compares times was 2 to 2.5 times faster. But searches that have to return whole rows are no faster and mostly slower with columns, because each row is rebuilt from five arrays: copying 25.3 million rows took 87.5 ms against 73.9 ms (1.18 times slower), the batched callback 1.13 times slower, the per-row virtual call 1.3 to 1.8 times slower, and the Q1 copy of one year of one monitor took 15.4 µs against 1.38 µs (11 times slower). Q3 and the Q1 view were unchanged within noise. The answer to the assignment's question of whether queries should shape the design is therefore yes: the right layout depends on what the query returns, and a single library that must serve both kinds of queries needs both layouts or a choice made per query.

Search	rows	columns	columns / rows
Q2 ozone > 0.030 (54%), copy	73.9 ms	87.5 ms	1.18
Q2 ozone > 0.030 (54%), callback	81.0 ms	91.8 ms	1.13
Q2 ozone > 0.070, virtual	47.4 ms	70.3 ms	1.48

Search	rows	columns	columns / rows
Q2 ozone > 0.030, virtual	77.6 ms	120 ms	1.55
Q1 one year, copy	1.38 µs	15.4 µs	11.2
Q1 3 hours, scan (no index)	18.0 ms	7.2 ms	0.40
Q1 one year, scan (no index)	19.0 ms	9.44 ms	0.50
Q1 one year, view	83 ns	83 ns	1.00 (timer resolution)
Q3 none / by hour / by monitor	5.08 / 5.91 / 29.3 ms	4.66 / 6.09 / 29.5 ms	0.92 / 1.03 / 1.01

Table 5. Column against row layout on M-A, final run on mains power, minimum of ten repeats, both layouts measured in the same session with the new tokenizer and indexed search. Only the ratios should be read: the absolute times in this session were higher than in other sessions (Section 9). The Q2 count is in Table 6.

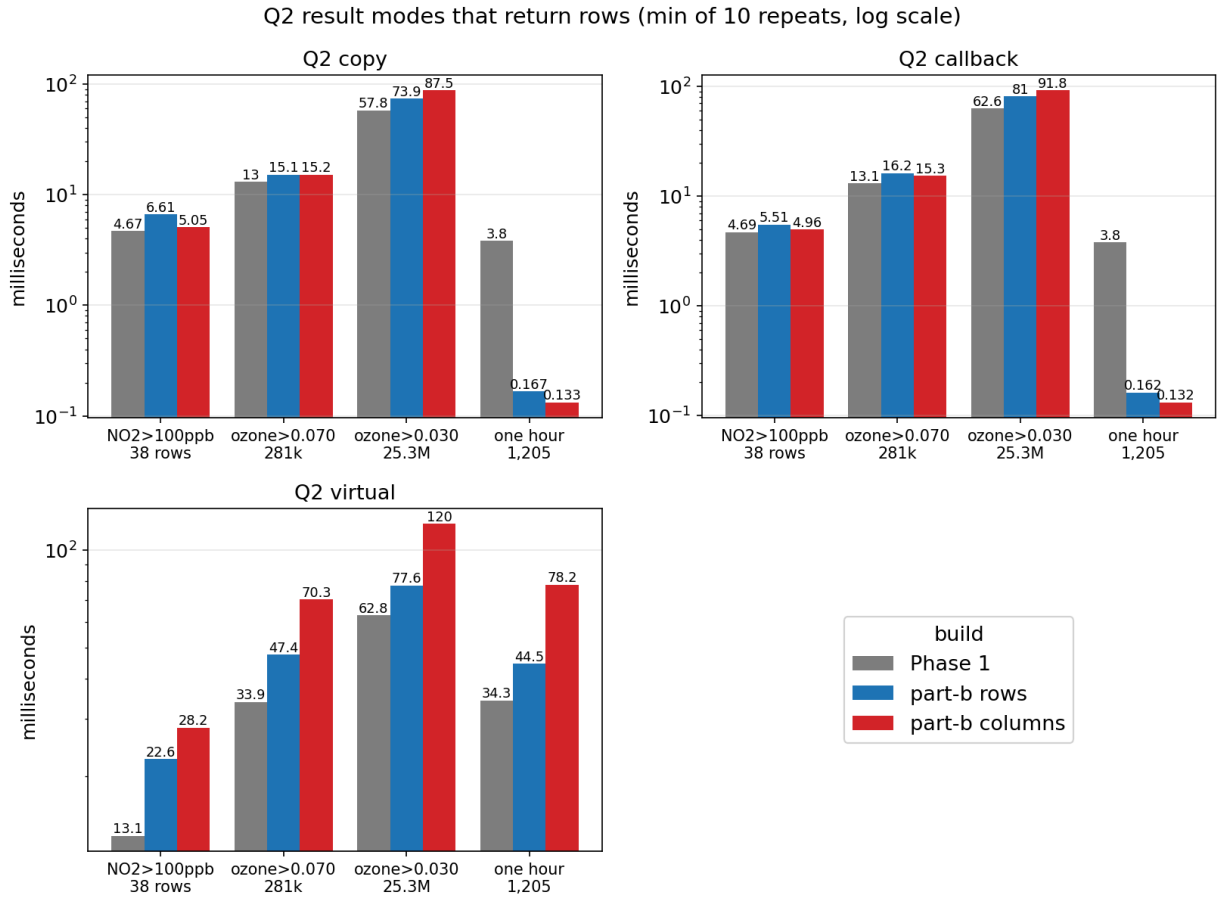


Figure 4. Q2 result modes for Phase 1, the row layout and the column layout. The Phase 1 bars come from an earlier session than the other two, and the final run's row-returning times were higher than in other sessions (Section 9), so compare the rows and columns bars with each other, not with Phase 1.

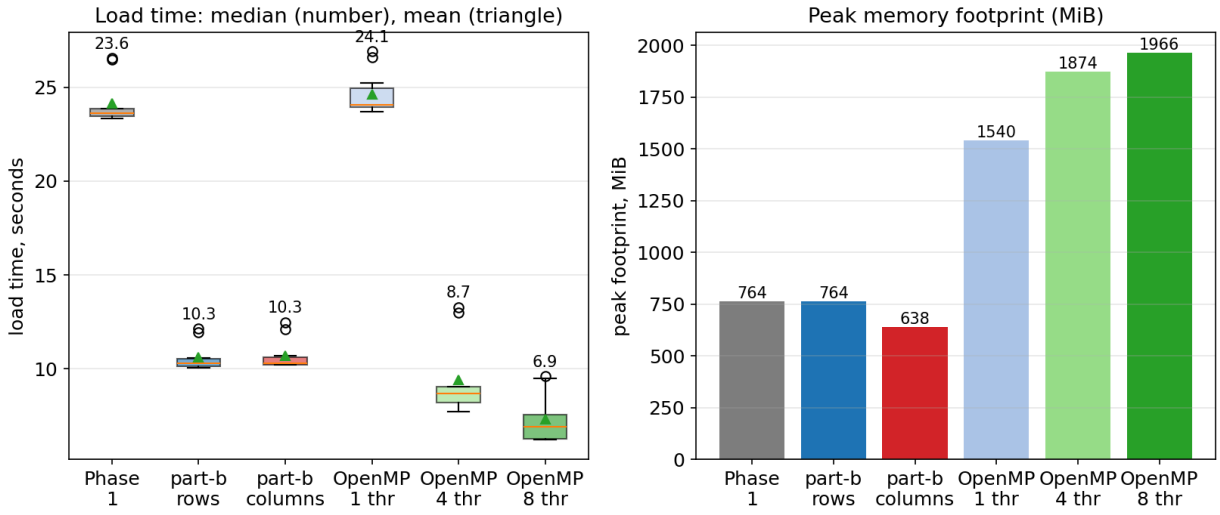


Figure 5. Interleaved full-dataset loads on M-A (ten rounds, mains power, one session including Phase 1) and peak memory.

6.3 Why the first column count was disappointing, and the fix

The first column count took 8.3 ms for 94 MB, about 11 GB/s, although the row layout, which reads six times more data, reached about 50 GB/s on its scan. The column loop was therefore not limited by memory. The loop added `(v >= lo) & (v <= hi)` to a 64-bit counter, and the wide counter, we believe, kept the compiler from using narrow SIMD lanes on 16-bit data; we inferred this from the speedup and did not inspect the generated assembly. We replaced the two comparisons with one unsigned comparison, `(uint16)(v - lo) <= (uint16)(hi - lo)`, which is valid when `lo <= hi`, and accumulated in a 32-bit counter that is added to the 64-bit total every 65,536 rows. The result is identical (281,349 rows and so on) and the loop sped up to about 34 GB/s. That is still below what the row scan reached (about 50 GB/s with the old loop, and 76 GB/s for the new row loop, 564 MB in 7.4 ms), so we cannot claim the column loop is now limited by memory; we did not investigate further. Because that change also helps the row layout, we gave the row layout the same loop to keep the comparison honest. Table 6 and Figure 6 separate the two effects: the loop alone makes the row count 1.3 to 1.5 times faster, and the layout adds another 2.7 to 3.1 times on top of it. The time of the column count then no longer depends on how many rows match (2.73 ms at 0.6% and at 54%).

Q2 count	Phase 1	rows, old loop	rows, new loop	columns, old loop	columns, new loop
ozone > 0.070 (0.6%)	11.4 ms	11.3	8.49	8.32	2.73
ozone > 0.030 (54%)	11.3 ms	11.3	7.62	8.36	2.73
NO2 > 100 ppb	4.49 ms	4.51	2.94	3.34	1.09
one hour (time-only)	3.80 ms	3.75	3.89	3.75	3.75

Table 6. Q2 count before and after the loop change, M-A, minimum of ten repeats. The old-loop columns come from the first Step 2 run, which began on battery power; a separate measurement made on mains power just before the change gave 8.33 ms (columns) and 11.32 ms (rows) for the 0.6% query with only ozone loaded, in agreement. The new-loop columns were measured on mains power.

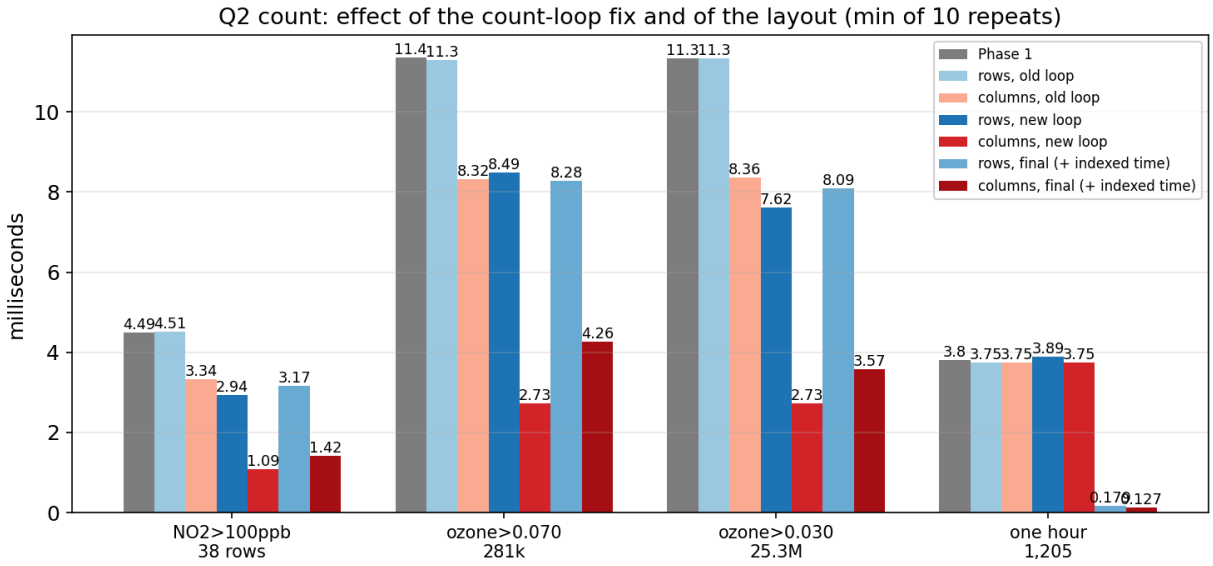


Figure 6. Q2 count on M-A for the four queries of Table 6, from Phase 1 through the old and new loop on both layouts to the final build (minimum of ten repeats). The "final" bars include the indexed time-range search; the 4.26 ms column bar for ozone > 0.070 is the outlier discussed with Table 7.

7. Phase 2 optimizations: single-pass tokenizer and indexed time search

Dhruv Jain replaced the tokenizer with a single forward scan over each line instead of two find calls per field, preserving the old behavior for quoted fields. A test compares every field from both 2024 files against the previous parser, and the full-dataset regression passes in both layouts. On W-1 the load of 2024 data (12,542,994 rows) fell from 7.58 s to 4.30 s (1.76 times) and the full dataset from 42.34 s to 24.66 s (1.72 times), with identical row counts and the same memory. On M-A the effect was larger: the serial full-dataset load fell from a median of 23.61 s (Phase 1) to 10.28 s (rows) and 10.32 s (columns), 2.3 times faster, with the same peak memory for the row layout. Since the profiles in Section 4.1 attributed about 78% of the load to splitting, a gain of this size is what we expected; the difference between the two machines (1.72 and 2.3 times) shows that the benefit depends on the machine, so ratios should not be carried from one machine to another.

He also made the time-only Q2 search use the per-monitor time-sorted blocks. For a time range that only partly overlaps a file, two binary searches narrow each monitor's block before the value test, and the resulting ranges are sorted by position so that copy and callback output keeps the previous order. Ranges that cover a whole file keep the original vectorized scan, and a control with a full-year range showed no slowdown (rows 9.67 to 9.59 ms, columns 1.57 to 1.55 ms on W-1). On W-1 a one-hour search with 1,205 results ran 38 to 43 times faster on the row layout and 33 to 40 times faster on the column layout, and a five-day search (140,298 results) 8 to 23 times faster. On M-A the one-hour search fell from the Phase 1 baseline of 3.80 ms to 0.179 ms on rows (21 times) and 0.127 ms on columns (30 times); copy and callback showed the same gain (0.13 to 0.17 ms). Table 7 summarizes the M-A numbers.

Search	Phase 1	rows (final)	columns (final)
Q2 one hour, count	3.80 ms	0.179 ms	0.127 ms
Q2 one hour, copy	3.80 ms	0.167 ms	0.133 ms
Q2 one hour, callback	3.80 ms	0.162 ms	0.132 ms

Search	Phase 1	rows (final)	columns (final)
Q2 ozone > 0.070, count	11.4 ms	8.28 ms	4.26 ms (re-runs 2.73 to 3.06)
Q2 ozone > 0.030, count	11.3 ms	8.09 ms	3.57 ms
Q2 NO2 > 100 ppb, count	4.49 ms	3.17 ms	1.42 ms

Table 7. Final Q2 results on M-A (minimum of ten repeats). The 4.26 ms for columns is an outlier: three reruns of the same query gave 2.73, 3.06 and 2.93 ms.

8. Phase 1 compared with Phase 2

Table 8 compares the designs on M-A. The largest gains came from algorithmic and data-layout changes, not from threads: a better tokenizer made the serial load 2.3 times faster, an index made the time-only search 21 to 30 times faster, and a better loop on a narrower layout made the count about 4 times faster. Eight threads made the load 3.5 times faster than the same code on one thread, but only 1.5 times faster than the serial load with the new tokenizer, and at 2.6 times the memory; the serial column count beat the threaded row count at every thread count we tried (the threaded code uses the original comparison loop, so this compares layout and loop together against threads, which is the combination a designer would choose between). The comparison supports a simple order of work: find where the time goes, reduce the work, and only then parallelize, since threads multiply whatever the serial code does. We did not combine threads with the new tokenizer and layout (Section 9), so we cannot say how much more the combination would give.

Measure (M-A)	Phase 1	Phase 2 best	Change
Full load, serial	23.61 s	10.28 s	2.3 times faster
Full load, OpenMP 8 threads (old tokenizer)	23.61 s	6.94 s	3.4 times faster
Peak memory	764 MiB	638 MiB (columns)	16% less
Peak memory, OpenMP load	764 MiB	1,966 MiB (8 threads)	2.6 times more
Time-only Q2	3.80 ms	0.127 ms (columns)	30 times faster
Q2 count, 0.6%	11.4 ms	2.7 to 4.3 ms (columns)	about 3 to 4 times faster
Q2 copy, 54%	57.8 ms	not improved	1.02 times Phase 1 (interleaved check, Section 9)

Table 8. Phase 1 against the best Phase 2 result for each measure.

9. Failures, negative results and limitations

We keep these because the assignment asks for them and because several changed what we did. The dataset download failed three times (two interruptions from our own shell handling and once a connection reset by the EPA server at 73% of one file); the download script re-downloads missing files and verifies the pinned checksums, so we ran it detached and re-ran it. Our first memory numbers were wrong (Section 4.2). The first OpenMP schedule gave no speedup (Section 5). The first column count ran far below memory speed because of how the loop was written (Section 6.3). Adding a `reserve` call to the column Q1 copy did not help (14.6 µs became 12.6 µs against 1.4 µs for rows); the cost is rebuilding each row from five arrays, about 1.4 ns per row against 0.15 ns per row for a bulk copy of 12-byte rows. A comparison script printed nothing because the shell did not split a command string into words, which was

a script error, not a measurement. Background benchmark jobs were killed when their shell ended and had to be started detached. The 4% load difference described in Section 3 was noise.

Several results are limited. First, OpenMP is not combined with the library: the OpenMP code is a separate copy of the Phase 1 design with the old tokenizer and the row layout, and the library is serial, so the combination of single-pass parsing, columns and threads was not built or measured. Second, ThreadSanitizer is not clean. Dhruv's GCC run completed all 1,010 synthetic checks but exited with status 66, and our Clang run on M-A reported 24 warnings across the two OpenMP tests (22 and 2) while all checks passed. Every one of the 24 pairs an access by a worker thread inside a parallel region with an access by the main thread to the same address after the region; none pairs two workers. This is the pattern one expects if ThreadSanitizer cannot see the barrier at the end of the parallel region as synchronization because the OpenMP runtime is not instrumented, and the design depends on disjoint writes (one result slot per file or segment) followed by a serial merge. That is our inference from the stack traces, not a proof; a clean confirmation needs an OpenMP runtime built with race-detection support, such as Archer [8][9][10]. Archer is part of the LLVM OpenMP sources, but we found no Archer library in our Homebrew installation (a file search under the LLVM and libomp directories) and did not build it from source. We therefore do not claim that the OpenMP code is race free. Third, timings of large copies and of Q3 on M-A vary from session to session: Phase 1 alone gave 58.5 to 64.9 ms for the 25-million-row copy and 3.8 to 5.0 ms for Q3 in consecutive measurements. In the final benchmark run, the row layout's copy, callback and virtual-call searches (other than the one-hour searches, which the indexed search had made much faster) measured 16% to 73% slower than the Phase 1 figures from earlier sessions (median 28%), which looked like a regression from the indexed-search change. To test this we built the commit before that change and compared Phase 1, that build and the current build, interleaved over four rounds on the same ozone data (run on battery power at 96% to 86%). The 25-million-row copy took 58.7 ms for Phase 1, 58.5 ms before the change and 59.6 ms now; the per-row virtual call 34.2, 34.1 and 34.2 ms; the ungrouped aggregate 3.92, 3.95 and 3.83 ms. There is no regression for these three. The batched callback is different: 63.0 ms for Phase 1, 58.2 ms before the change and 65.9 ms now, a consistent 13% slowdown against the previous build and 5% against Phase 1. We traced it to code generation, not to the algorithm, by experiment. The indexed search added a second loop in the scan function, so the compiler inlined the callback's batching code twice. Moving the temporary vector into the new branch made the callback much worse (112 ms), and restructuring the scan so that both cases share one loop, and therefore one inlined copy, made it faster than before (53.7 ms for rows, a 19% gain over the current code; for columns 85.1 ms became 63.9 ms, 25% faster). The same change made the copy slower (by 2% for rows and 11% for columns), so it is a trade between result modes rather than a free fix. We kept the code that all our measurements were taken with and did not adopt the change; it is recorded as a patch with our results. Whatever the compiler decides about inlining here changes the time of a result mode by 10 to 25%, which is a reason to treat differences of that size in the row-returning modes with caution. The high times of the final run itself remain unexplained. We tested four explanations and none reproduced it: loading both pollutants instead of only ozone gave 59.4 ms for the current build against 57.7 ms for Phase 1 on the 25-million-row copy; five minutes of sustained OpenMP loads before measuring (to heat the fanless laptop) left the copy at 58.8 ms, the same as on the cool machine (59.4 ms); and rebuilding the commit before the indexed-search change gave the same copy time as the current code (58.5 and 59.6 ms). A fourth test, with the charger connected, also did not reproduce it: the copy measured 58.0 to 59.0 ms (Phase 1 57.7 to 58.3 ms), although the laptop reported that it was attached but not charging, so we did not test the state of actively charging. We therefore cannot explain the final run's high row-returning times; the conclusions that rest on that run use ratios between layouts measured in the same session (Table 5), and the Phase 1 comparison for these modes rests on the interleaved check above. For this reason we

claim only the large changes (the load, the count and the time-only search), and treat the ratios between layouts within one session (Table 5) as more reliable than absolute times across sessions. Fourth, M-A is a laptop and Dhruv's numbers are from another machine and operating system, so absolute times are not portable. Fifth, Q4 was not benchmarked.

10. Conclusions and recommendations

First, measure before designing: the profile showed that line splitting, not input or storage, dominated the load, and replacing the tokenizer (a few dozen lines) gave a 2.3 times speedup on one thread at no cost in memory, while eight threads gave 3.5 times on the unmodified code at 2.6 times the memory; one thread with the new tokenizer was only 1.5 times slower than eight threads with the old one. Second, choose the layout by what queries return. Columns are the right layout for scans that read one field and the wrong one for queries that return rows, so if both are needed, store both or choose per query. Third, exact integer storage is not an optimization but a correctness requirement for threshold queries on this data. Fourth, a library boundary built with a private implementation and hidden symbols can be free in speed, and linkage (static or shared) should be chosen for deployment, not speed. Fifth, when comparing builds on a laptop, interleave runs, use mains power, report medians and keep outliers. Sixth, on macOS use the physical footprint, not resident size, for memory. The most valuable next step would be combining the OpenMP code with the single-pass tokenizer and the column layout, and then bounding the load's memory by writing parsed rows straight into one pre-sized array instead of per-file vectors.

11. Individual contributions

The commit history of the repository records who wrote what, and the following summary follows it.

Vandan Shah created the repository and the build setup (shared CMake configuration, the part-a and part-b skeletons, the benchmarking, plotting and packaging scripts, and the dataset download script with pinned checksums) and wrote the Phase 1 library: the value types, the tokenizer and the three file readers, the monitor registry and dictionaries, the row store with its segment and block indexes, the facade, the driver program, and the Phase 1 tests.

Neeraja Buch fixed the macOS memory measurement, ran the Phase 1 baselines on the full dataset and the Phase 1 experiments, profiled the load, and produced the Phase 1 graphs. She wrote the Phase 2 step 2 library and application (the facade with a private implementation, hidden symbols, static and shared builds, the column layout, the strided column view, and the tests for both layouts), found and fixed the column count loop, ran all final M-A benchmarks and the interleaved load comparison, wrote the comparison graphs and the ThreadSanitizer analysis on M-A, and wrote the lab notes from which this report is drawn.

Dhruv Jain wrote the single-pass tokenizer with its test, the indexed time-range search for both layouts with its tests, and the OpenMP implementation in part-b/omp (parallel searches, parallel loading with deterministic merging, the schedule fix, the OpenMP tests and sanitizer runs), and measured the tokenizer, indexed search and OpenMP scaling on W-1.

References

All web addresses were fetched on 11 October 2026 and returned the expected page (the ACM and IEEE DOI links refuse or defer automated requests, so [8] and [9] were checked through the Google Research and author-hosted pages listed below, which show the same title, authors, venue and page numbers).

[1] U.S. Environmental Protection Agency, “Air Quality System (AQS) pre-generated data files,” https://aqs.epa.gov/aqsweb/airdata/download_files.html (lists the hourly ozone (44201) and NO2 (42602) files, one zip per pollutant and year).

[2] U.S. Environmental Protection Agency, “AQS data file formats,” section 6, “Hourly Data Files,” https://aqs.epa.gov/aqsweb/airdata/FileFormats.html (field definitions; a monitor is identified by state, county, site number, parameter code and parameter occurrence code).

[3] Apple Inc., XNU kernel header osfmk/mach/task_info.h (struct task_vm_info, fields phys_footprint and ledger_phys_footprint_peak), https://github.com/apple/darwin-xnu/blob/main/osfmk/mach/task_info.h; see also “phys_footprint,” Apple Developer Documentation, https://developer.apple.com/documentation/kernel/task_vm_info_data_t/1553210-phys_footprint.

[4] OpenMP Architecture Review Board, “OpenMP Application Programming Interface,” version 5.2, November 2021, the schedule clause of loop constructs, https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf (listed at https://www.openmp.org/specifications/).

[5] H. Sutter, *Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions*, Addison-Wesley, 2000, ISBN 0-201-61562-2, Items 26 to 30 (minimizing compile-time dependencies, compilation firewalls and the pimpl idiom).

[6] U. Drepper, “What Every Programmer Should Know About Memory,” Red Hat, Inc., 21 November 2007, https://www.akkadia.org/drepper/cpumemory.pdf (cache behavior of sequential and non-sequential memory access).

[7] M. Stonebraker, D. J. Abadi, A. Batkin, X. Chen, M. Cherniack, M. Ferreira, E. Lau, A. Lin, S. Madden, E. O’Neil, P. O’Neil, A. Rasin, N. Tran and S. Zdonik, “C-Store: A Column-oriented DBMS,” *Proceedings of the 31st International Conference on Very Large Data Bases (VLDB)*, Trondheim, Norway, 2005, pp. 553-564, https://vldb.org/conf/2005/papers/p553-stonebraker.pdf.

[8] K. Serebryany and T. Iskhodzhanov, “ThreadSanitizer: data race detection in practice,” *Proceedings of the Workshop on Binary Instrumentation and Applications (WBIA ’09)*, 2009, pp. 62-71, https://research.google/pubs/threadsanitizer-data-race-detection-in-practice/ (DOI 10.1145/1791194.1791203).

[9] S. Atzeni, G. Gopalakrishnan, Z. Rakamaric, D. H. Ahn, I. Laguna, M. Schulz, G. L. Lee, J. Protze and M. S. Muller, “ARCHER: Effectively Spotting Data Races in Large OpenMP Applications,” *Proceedings of the 30th IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 53-62, https://soarlab.org/publications/2016_ipdps_agralslpm/ (DOI 10.1109/IPDPS.2016.68).

[10] LLVM Project, “Archer” (OpenMP tools), https://github.com/llvm/llvm-project/tree/main/openmp/tools/archer (an OMPT tool that annotates OpenMP synchronization for ThreadSanitizer).